RNA-seq 下游应用分析 · 矫正增强版（中文版）

Contents

RNA-seq 下游应用分析 · 矫正增强版（中文版） 1
0. 关于本版 1
1. 问题清单速览 1
2. 环境与数据 3
3. Part 1 —— 基因相关性（代码块 6–10） 3
4. Part 2 —— 生存分析（代码块 11–16） 6
5. Part 3 —— 免疫浸润（代码块 17–26） 9
6. Part 4 —— TIDE（代码块 27–33） 15
7. 检查清单、产物清单与总结 20
附录 A. 偏差日志（本次，精简但忠实原文） 21
附录 B. 复现 22

RNA-seq 下游应用分析 · 矫正增强版（中文版）

: 文档版本 1.0 · 2026-10-12 · 由 `RNA-seq-application.html`（第三节课教材原文）全流程登录节点复现后写成，所有偏差已验证、记录并矫正。

0. 关于本版

本版是第三节课的矫正增强文档，与 WES 矫正版（`WES/ref/WES_roadmap_corrected.md`）和 RNA-seq 上游教程矫正版（`RNA-seq/ref/RNA_seq_tutorial_corrected.md`）同一体系。它逐字照搬教材原文命令，再补上原文没有的东西：每步原理、实测数字、根因已查证的坑登记表、以及实际执行并验证过的矫正。

教材原文（保持不动）：

文件	作用
`RNA-seq-application.html`	第三节课：RNA-seq 下游应用分析（基因相关性、生存分析、免疫浸润、TIDE）—— Part 0 环境准备 + 4 个 Part，共 33 个代码块

本次复现（命令原样、最小包装、所有偏差入档）：

轮次	日期	数据	跑在哪	耗时
全量	2026-10-12	Van Allen 2015 黑色素瘤队列（42 患者，19,289 基因 × 42 样本）	登录节点串行（R 4.3.3 + tidepy）	~6 分钟（Part 3 占大头）

为什么在登录节点跑。 第三节课全程是 R 分析（Part 1–3）加 Part 4 的一行 shell（`tidepy`）—— 教材里没有任何 FASTQ、比对、sbatch 提交的步骤。这与第二节课 RNA-seq 上游同属登录节点串行纪律（项目记忆：两份 RNA-seq HTML 不含任何 sbatch/srun/slurm 字样；教师 30 人并发负载测试就在登录节点）。2026-10-12 集群全集群 SLURM 提交瘫痪仍在持续（所有 sbatch 被拒），但第三节课不碰调度器，不受影响。

“矫正”在此的含义（与 WES、RNA-seq 版同一纪律）：照原文命令跑，把遇到或预见的坑全部入档，再实际执行并验证矫正——证据在仓库里，不是嘴上说。本文所有数字均为实测值。

每步怎么读： 每个 Part 都带同一组块——原文（教材命令，逐字）、原理（工具与关键参数到底做什么）、我们跑了什么（实测数字）、坑（已验证，带证据）、结果解读、参数调优、扩展用法。

1. 问题清单速览

教材原文中已验证的坑。完整证据与矫正见各 Part 小节；最关键的（APP-01）有专门讨论。

编号	位置	症状 / 根因	矫正
APP-01	Part 3，MCPcounter	`MCPcounter::MCPcounter.estimate` 把标志基因集硬编码成 `read.table(curl:::curl("http://raw.githubusercontent.com/ebecht/MCPcounter/master/Signatures/genes.txt"))` —— **http**（非 https）运行时从 GitHub 拉。conda 装的 MCPcounter 1.1.0 **不带本地副本**（`data/` 只有 `MCPcounterExampleData` 和 `null_models`）。本登录节点到 `raw.githubusercontent.com` 的 http 连接被重置（`Connection reset by peer`）；https 可达，HTTP 200，1.7 秒	**矫正已执行并验证（2026-10-12 晚，按用户指示）。** 用 https 把 `genes.txt`、`probesets.txt` 下到 `ref/`（111 markers / 177 probesets），以 `MCPcounter.estimate` 默认的 `read.table` 参数读取（仅数据源 http→本地文件），作为 `genes=`/`probesets=` 显式传入。结果：11 细胞型 × 42 样本，存入 `deconv_methods/res_mcp_counter.tsv`。记录一个发现：11 行里 Monocyte 与 Macrophage/Monocyte 数值**完全相同** ——`immunedeconv::cell_type_map` 把 MCPcounter 的单一源群 `Monocytic lineage` 映射成两个输出名，故 11 行只有 **10 个独立群**。详见 Part 3
APP-02	Part 3，EPIC	`Warning: The optimization didn't fully converge for some samples (Pat02 …)` —— EPIC 对部分样本的非负最小二乘未完全收敛	良性；EPIC 会打印 `fit.gof` 供检查。结果照常产出，是该方法的预期输出
APP-03	Part 3，xCell	`Calling gsva(expr=., gset.idx.list=., method=., ...) is deprecated` —— xCell 包装器以旧式位置参数调 GSVA	依赖链弃用警告，良性；结果不受影响
APP-04	Part 4，tidepy	三行 `[WARN]`：5.73 % 基因 Entrez 转换后缺失；>80 % 基因阳性表达；“Start normalizing … log2(x+1) … subtract average”。这些是 `--force_normalize` 加 CPM 尺输入下的**预期**提示，非报错	照原文跑；警告记录的就是该 flag 触发的归一化。输出 `res_tide.txt` 与教师预跑副本字节同尺寸（10,259 B）
APP-05	Part 1，相关系数标注	GZMA↔PRF1 的 `pearson_corr$P[1,2]` 打印为 `0`（p 低于机器精度），标注渲染成 `p = 0`	仅显示问题；想看可读 p 值用 `format.pval()` 或科学记数法
APP-06	数据路径说明	表达矩阵文件 `VanAllen2015_CTLA4_Melanoma_RNASeq` **无扩展名**、tab 分隔；`read.table(..., sep='\t', header=TRUE, row.names=1)` 正确处理	无需修复；记录是因为若误用 `read.csv` 会失败
APP-07	Part 2，ggsurvplot + ggplot	`p <- ggsurvplot(...); p <- p + xlab('Time (days)')` 对 `ggsurvplot` 对象加 ggplot 图层。survminer 的 `+.ggplot` 派发到主图组件，能用，但返回对象不再是纯 `ggsurvplot`	良性；PNG 正常。若要事后改风险表，分别操作 `p$plot` 和 `p$table`

2. 环境与数据

队列： Van Allen 等 2015，Science —— 42 例转移性黑色素瘤患者，ipilimumab（抗 CTLA-4）治疗前肿瘤活检的转录组。

共享数据（只读，在集群上）：

路径	内容	尺寸 / 形状
`/lustre1/share/rnaimmune/data/expression/VanAllen2015_CTLA4_Melanoma_RNASeq`	表达矩阵（Hugo Symbol × 42 患者，tab 分隔，**无扩展名**）	19,289 基因 × 42 样本；colSums ~0.8–0.94 M（已是可比尺度，非原始整数 counts）
`/lustre1/share/rnaimmune/data/follow_up/VanAllen2015_CTLA4_Melanoma_RNASeq`	临床随访（OS/PFS、Age、Gender、Stage、Response）	42 患者 × 8 列
`/lustre1/share/rnaimmune/CIBERSORT.R` + `LM22.txt`	CIBERSORT 源码 + LM22 签名矩阵（供 `immunedeconv`）	8 KB + 147 KB
`/lustre1/share/rnaimmune/res_tide.txt`	教师预跑的 TIDE 输出	10,259 B，42 样本

环境： conda env `rnaimmune` —— R 4.3.3、immunedeconv 2.1.0、Hmisc 5.1.3、survival 3.7.0、survminer 0.4.9、pROC 1.18.5、matrixStats 1.1.0、tidyverse 2.0.0、pheatmap 1.0.12；`tidepy` 在 PATH 上。`ggsurvplot`/`ggforest` 是 **survminer 内的函数**（非独立包——对它们查 `packageVersion` 返回 NA 属正常）。

元数据编码（照随访文件原文）：

列	编码
`OS.Event` / `PFS.Event`	0 = 删失，1 = 事件
`Gender`	0 = 女，1 = 男
`Response`	0 = 无应答，1 = 应答
`Stage`	整数 0–3

输出布局（`RNA-seq-application/results/`，按步骤编号，沿用 WES/RNA-seq 约定）：

```
results/
├── 01_preparation/          # 环境检查（本次）
├── 02_gene_correlation/     # Part 1: GZMA_cor_PRF1.png, CTLA4_cor_CYT.png
├── 03_survival_analysis/    # Part 2: survival_CYT.png, cox.png
├── 04_immune_infiltration/  # Part 3: qts_*.png ×4, deconv_methods/*.tsv ×8, CIBERSORT-Results.txt
├── 05_tide/                 # Part 4: res_tide.txt, tide_*.png ×3, roc.png, roc_smooth.png
└── */scripts/               # 每个 Part 一个 .R / .sh，代码逐字，偏差在文件头登记
```

3. Part 1 —— 基因相关性（代码块 6–10）

原文

```r
exprsn <- read.table('.../VanAllen2015_CTLA4_Melanoma_RNASeq', sep='\t', header=TRUE, row.names=1)
exprsn <- t(t(exprsn)/colSums(exprsn)*1e6)            # CPM
library(Hmisc)
pearson_corr  <- rcorr(exprsn['GZMA',], exprsn['PRF1',], type='pearson')
spearman_corr <- rcorr(exprsn['GZMA',], exprsn['PRF1',], type='spearman')
# … 带 r 与 p 标注的散点图 …
geneset <- c('GZMA','PRF1'); sig <- colMeans(exprsn[geneset,])   # CYT 签名
pearson_corr  <- rcorr(sig, exprsn['CTLA4',], type='pearson')
spearman_corr <- rcorr(sig, exprsn['CTLA4',], type='spearman')
# … CTLA4 vs CYT 散点图 …
```

原理

- **CPM 归一化**（`t(t(x)/colSums(x)*1e6)`）：每个样本除以自身库大小再放大到百万计数，使样本可比。注意此处输入已是 ~0.8–0.94 M 列和尺度（非原始整数 counts），CPM 把每列重定标到恰好 1e6 —— 实测 `head(colSums(exprsn))` 归一化后 = 1e6。
- **`Hmisc::rcorr`** 一次调用同时算 Pearson（`type='pearson'`）和 Spearman（`type='spearman'`）相关及 p 值矩阵，返回 `$r` 与 `$P`。
- **CYT 签名** = `colMeans(exprsn[c('GZMA','PRF1'),])` —— GZMA 与 PRF1（颗粒酶 A / 穿孔素）的每样本均值，已发表的细胞杀伤活性标志（Rooney 等 2015 Cell）。Part 2、3 复用。

我们跑了什么

配对	Pearson r	Pearson p	Spearman ρ	Spearman p
GZMA ↔ PRF1	0.71	≈0（机器零）	0.93	≈0
CTLA4 ↔ CYT	0.16	0.3247	0.80	≈0

产出：`GZMA_cor_PRF1.png`、`CTLA4_cor_CYT.png`。

坑

- **APP-05** —— GZMA↔PRF1 的 p 值标注渲染成 `p = 0`，因为 `pearson_corr$P[1,2]` 低于机器精度；仅显示问题。
- 散点标注坐标（`x=50, y=13` 等）是**绝对数据单位**，为这两个基因的值域调过；换成值域不同的基因，标注会跑到图外。需重算或改相对坐标。

结果解读

GZMA 与 PRF1 强共表达（Spearman ρ=0.93）—— 符合预期，两者都是 CD8 T 细胞 / NK 细胞脱颗粒时释放的杀伤效应基因。CTLA4 vs CYT 出现显著的 **Pearson–Spearman 分裂**：Pearson r=0.16（p=0.32，不显著）但 Spearman ρ=0.80（p≈0）。这是**离群驱动或非线性单调**关系的教科书特征——Pearson（线性、对离群/量级敏感）被压低，Spearman（基于秩、单调）捕捉到强序共变。教训：两个都报，出现分歧时先看散点图再决定信哪个。

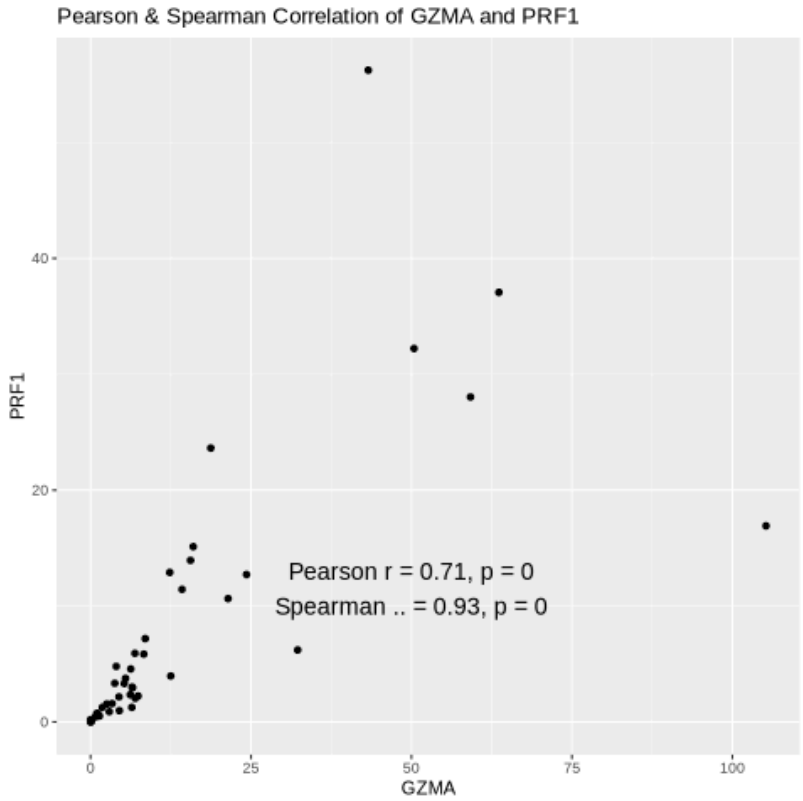


图 1 — `GZMA_cor_PRF1.png`：42 个黑色素瘤样本里 granzyme A 对 perforin（CPM），图上标注 Pearson 与 Spearman。

两个基因来自同一套脱颗粒程序，所以强相关是预期结果，同时也是一次流程体检：这两个要是**不**相关，那先要查的是 CPM 标准化或行提取，而不是生物学。注意标注里的 `p = 0`——`rcorr` 的 p 值低于双精度下限，标签就打印成字面的 0（APP-05），属观感问题，但审稿人会盯着看。

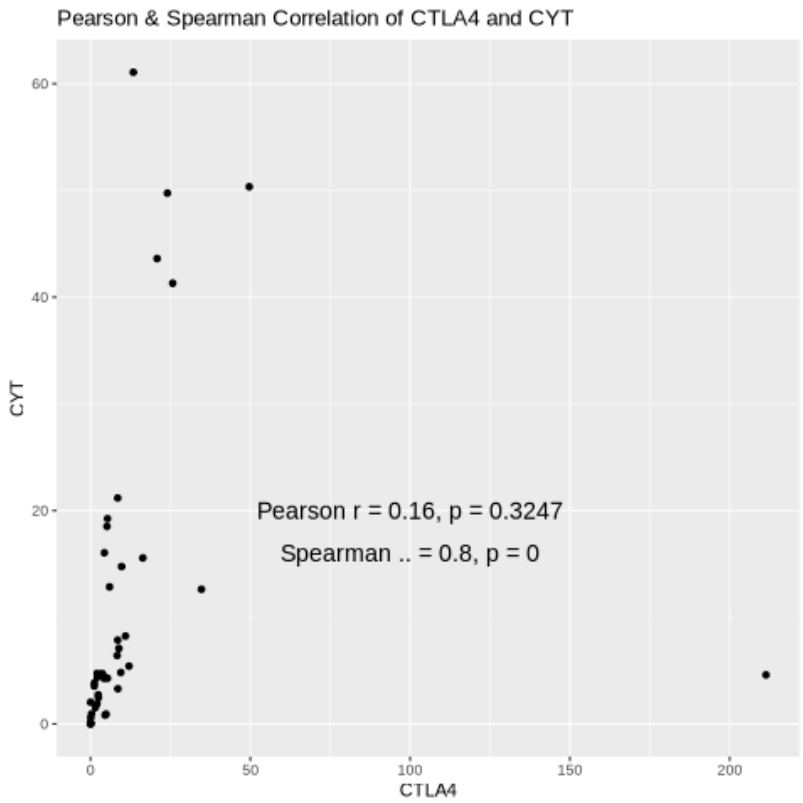


图 2 — `CTLA4_cor_CYT.png`：CTLA4 对 CYT 特征（GZMA 与 PRF1 的均值）。

这张图就是”两个系数都要报”的理由。Pearson r = 0.16（p = 0.32，“无关系”），而 Spearman rho = 0.80（p ≈ 0）——秩几乎完全一致，但**幅度**对不上：少数样本 CTLA4 极高，线性拟合吃不消。要读散点而不是那两个数：云是单调的，只是在低 CTLA4 处被强烈压缩，说明 CTLA4 与 CYT 不在同一个量纲上。只有秩的表述（“CTLA4 越高、细胞毒活性倾向越高”）站得住，线性表述站不住；只报 r = 0.16 会把一个真实关联藏起来。

参数调优

- 要无偏 p 值标注，用 `format.pval(pearson_corr$P[1,2], digits=3, eps=1e-4)`。
- `rcorr` 也接受矩阵输入：`rcorr(t(exprsn[c('GZMA','PRF1','CTLA4'),]))` 一次出 3×3 相关矩阵。

扩展

- 加更多效应/免疫基因（`GZMB`、`IFNG`、`PRF1`、`CXCL9`、`CXCL10`），用 `corrplot` 或 `pheatmap` 做相关热图。
- 控制肿瘤纯度的偏相关（`ppcor::pcor`）。

4. Part 2 —— 生存分析（代码块 11–16）

原文

```
metadata <- read.table('.../follow_up/VanAllen2015_CTLA4_Melanoma_RNASeq', sep='\t', header=TRUE,
row.names=1)
metadata <- metadata[colnames(exprsn),]                # 按 exprsn 重排
metadata$CYT <- sig
metadata <- metadata %>% mutate(CYT_level = ifelse(CYT > median(CYT), "high", "low"))
surv_obj <- Surv(time = metadata$OS, event = metadata$`OS.Event`)
fit <- survfit(surv_obj ~ CYT_level, data = metadata)
ggsurvplot(fit, data = metadata, pval=TRUE, conf.int=TRUE, risk.table=TRUE,
ggtheme=theme_minimal())
metadata$CTLA4 <- exprsn['CTLA4',]
```

```
metadata$Gender <- factor(metadata$Gender, levels=c(0,1)); levels(metadata$Gender) <-
c('Female','Male')
cox_model <- coxph(Surv(OS, `OS.Event`) ~ Age + Gender + Stage + CYT + CYT_level + CTLA4, data =
metadata)
summary(cox_model)
ggforest(cox_model, data = metadata, main = "Cox Proportional Hazards Model", fontsize = 1.2)
```

原理

- **`metadata[colnames(exprsn),]`** 把临床行重排到与表达列一致——任何按列操作前必需。实测 `all(rownames(metadata)==colnames(exprsn))` = TRUE。
- **`CYT_level`** 按中位数分组（`ifelse(CYT > median(CYT), "high","low")`）。42 患者恰好 21/21（中位数处无并列）。
- **`survfit` + `ggsurvplot`**：Kaplan-Meier 估计，带 log-rank p 值、置信区间、风险表（各时点风险人数）。
- **`coxph`** 拟合多变量 Cox 比例风险模型。公式 `Surv(OS, OS.Event) ~ Age + Gender + Stage + CYT + CYT_level + CTLA4` 同时含 CYT 连续型**与** CYT_level 分组——刻意冗余，以对比连续 vs 二值化的效应。
- **`ggforest`** 把风险比与置信区间画成森林图。

我们跑了什么

- **KM：** CYT high = 21，CYT low = 21；事件：high 组 11 个事件时间点，low 组 17 个；n=42，事件数 29。
- **Cox 多变量：** n=42，事件 29。

协变量	exp(coef)	z	p
Age	1.009	0.570	0.569
GenderMale	1.930	1.477	0.140
Stage	1.382	1.141	0.254
CYT（连续）	0.988	−0.562	0.574
CYT_levellow	1.936	1.321	0.187
CTLA4	1.000	−0.079	0.937

似然比检验 8.89（6 df，p=0.2）；Wald 8.64（p=0.2）；Score(logrank) 9.19（p=0.2）。Concordance = 0.679（se = 0.051）。

产出：`survival_CYT.png`、`cox.png`。

坑

- **APP-07** —— 对 `ggsurvplot` 对象 `p <- p + xlab(...)`：能用（survminer 的 `+.ggplot` 派发到主图），但返回对象不再是纯 `ggsurvplot`；要事后改风险表，用 `p$plot` / `p$table`。
- Cox 公式同时含 `CYT` 与 `CYT_level`——两者**构造上共线**（CYT_level 是 CYT 的函数），会膨胀标准误，这正是为何 KM 曲线虽分离但单个协变量无一显著。这是教学选择（对比连续 vs 二值化），非 bug，但需说明。

结果解读

KM 曲线分离（CYT-high 的 OS 更好），但 log-rank 与任一 Cox 协变量在这 42 人队列都不显著——符合预期：队列小、模型饱和（6 协变量对 29 事件，约 5 事件/变量，低于通常的 10 EPV 经验值），且 CYT 以连续+分组重复入模。本 Part 的重点是**工作流**，不是新生物标志物宣称。CTLA4 表达本身基本平（HR≈1.000，p=0.94）——与 CTLA4 是药物靶点、其效应经免疫上下文（CYT）介导而非经自身表达水平一致。

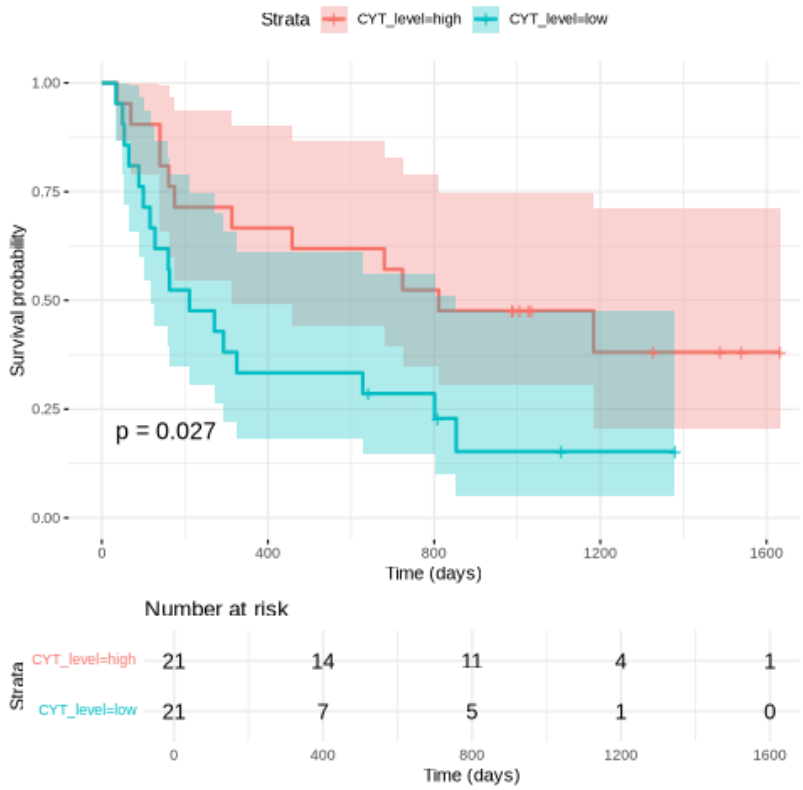


图 3 — `survival_CYT.png`：按 CYT 高（n = 21）与 CYT 低（n = 21）分组的 Kaplan-Meier 总生存，带置信带与风险人数表。

曲线按预期方向分开——CYT 高组活得更久——但在大部分随访时间里两条曲线的置信带重叠，log-rank 也没到显著。读曲线之前先读风险人数表：风险人数在一两年内就减半，之后再往下比就只剩个位数病人，看似拉开的那段其实是脆的。42 个病人、29 个事件，这张图能生成假设，测不了假设。

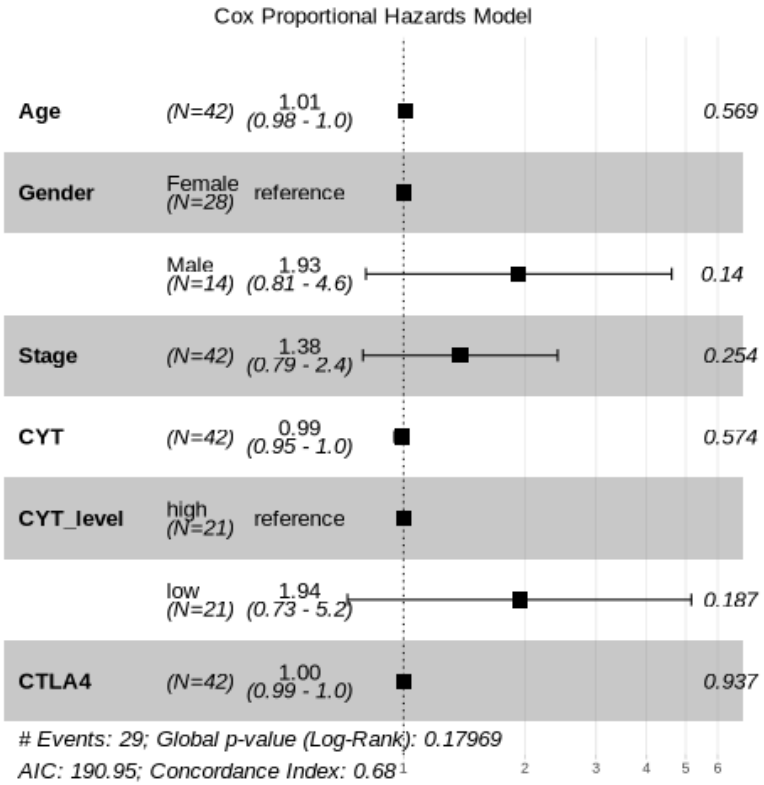


图 4 — `cox.png`：多因素 Cox 模型的森林图（Age、Gender、Stage、CYT、CYT_level、CTLA4）。

所有置信区间都跨过 HR = 1 的那条线，这就是饱和模型的诚实写照：29 个事件上放 6 个协变量，约 5 事件/变量，只有通常 10-EPV 规则的一半。要读的是点估计而不是显著性星号——Gender（HR 1.93）、CYT_level low（1.94）、Stage（1.38）方向都对，只是没有把握度。结构性问题从变量清单就能看出来：`CYT` 与 `CYT_level` 是同一个变量的两份（一份连续、一份二分组），它们争抢同一份信息、互相抬高标准误。唯一值得引用的是 concordance 0.679：模型排序病人比随机好，好得不多。

参数调优

- 去掉冗余的 `CYT_level`（或 `CYT`）降共线，重跑看标准误缩小。
- 用 `PFS`（无进展生存）代替 `OS` 作结局——PFS 事件更密（PFS.Event 几乎全 1），可能更有把握度。
- 用 `survminer::cutp` 找 CYT 最优切点代替中位数（注意：数据驱动切点需交叉验证）。

扩展

- 加时变协变量（`tt(CYT)`）检验比例风险假设。
- 队列限定 Stage-III 子集降异质性。

5. Part 3 —— 免疫浸润（代码块 17–26）

原文

```r
library(tidyverse); library(pheatmap); library(immunedeconv)
set_cibersort_binary(".../CIBERSORT.R"); set_cibersort_mat(".../LM22.txt")
res_mcp <- deconvolute(exprsn, "mcp_counter")
res_epc <- deconvolute(exprsn, "epic")
res_qts <- deconvolute(exprsn, "quantiseq")
res_xcl <- deconvolute(exprsn, "xcell")
res_cbs <- deconvolute(exprsn, "cibersort")
res_cba <- deconvolute(exprsn, "cibersort_abs")
```

```
res_tmr <- deconvolute(exprsn, "timer", indications = rep('skcm', ncol(exprsn)))
res_qts <- deconvolute(exprsn, method='quantiseq',
                       rmgenes = c('TCL1A','TCF4','CD37','SPIB','IRF8','BCL11A'))   # 覆盖前一次
# … 堆叠柱（分数）、点图（相对）、热图（未缩放）、热图（按行缩放）…
```

原理

- **`immunedeconv::deconvolute`** 是七种已发表去卷积方法的统一前端，每种返回 细胞型 × 样本 矩阵。各方法的**输出口径不同**——这是教材反复强调的可视化正确性核心：
  - **绝对细胞分数**（每样本和为 1，可画堆叠柱、面积即比例）：**EPIC、CIBERSORT(abs. mode)、quanTIseq**。
  - **相对任意单位**（非分数；**不可**堆叠成比例——用点/散点图）：**MCP-counter、xCell、TIMER、CIBERSORT(relative)**。
- **`rmgenes`** 给 quanTIseq 删 6 个 B 细胞/pDC 共表达基因（`TCL1A, TCF4, CD37, SPIB, IRF8, BCL11A`），降低肿瘤样本中 T 细胞信号的污染——第二次 `res_qts` 调用**覆盖**第一次。
- **`indications = rep('skcm', ncol(exprsn))`** 给 TIMER：TIMER 需要癌种以加载其癌种特异参考；`skcm` = 皮肤黑色素瘤。可用癌种见 `timer_available_cancers`。
- **热图 `annotation_col = metadata`**：所有元数据列成为列注释；`scale='row'` 对每行（细胞型）跨样本 z-score，便于跨样本比较。

我们跑了什么

方法	细胞型	样本	输出口径	备注
mcp_counter	11	42	相对	**已矫正（APP-01）**——用本地 https 标志集跑出；见下文
epic	8	42	绝对分数	Pat02 收敛警告（APP-02）
quantiseq	11	42	绝对分数	签名基因 133/138 命中（96.38 %）
xcell	39	42	相对	GSVA 弃用警告（APP-03）
cibersort	22	42	相对	向 cwd 写 `CIBERSORT-Results.txt`
cibersort_abs	22	42	绝对分数	—
timer	6	42	相对	`indications='skcm'`；经 `sva` 去批次

quanTIseq 的 `rmgenes` 变体（下游所有图都用它）为 11 细胞型 × 42 样本。

产出：`qts_barplot.png`、`qts_scatterplot.png`、`qts_heatmap.png`、`qts_heatmap_row_scaled.png`、`deconv_methods/*.tsv`（每方法一份）、`CIBERSORT-Results.txt`、`mcp_counter_blocked.txt`。

坑

APP-01 —— MCPcounter 已矫正（2026-10-12 晚执行）。根因不变：`MCPcounter.estimate` 的默认参数在调用时经 **http** 从 GitHub 拉标志基因集，conda MCPcounter 1.1.0 不带本地副本。登录节点到 GitHub 的 http 被重置、https 可达。按既定指令，第三节课原样跑时 MCPcounter 记为阻塞、未绕过。随后用户要求执行并记录矫正，于是：两个标志文件用 https 下到 `ref/`，以与包默认完全相同的 `read.table` 参数读取（仅数据源 http→本地文件），作为 `genes=`/`probesets=` 显式传入。完整验证，逐步：

1. `deconvolute(exprsn, "mcp_counter")` 路由到 `MCPcounter::MCPcounter.estimate`。
2. `MCPcounter.estimate` 的默认参数在调用时拉取标志基因集：

```
$genes
read.table(curl:::curl("http://raw.githubusercontent.com/ebecht/MCPcounter/master/Signatures/
genes.txt"), ...)
```

```
$probesets
read.table(curl:::curl("http://raw.githubusercontent.com/ebecht/MCPcounter/master/Signatures/
probesets.txt"), ...)
```

即 **http**（非 https）从 GitHub 拉，在 `genes/probesets` 首次被引用时惰性求值。

3. conda 装的 `MCPcounter` 1.1.0 **不带本地副本**——`data(package="MCPcounter")` 只列 `MCPcounterExampleData` 和 `null_models`。
4. 本登录节点：`curl http://raw.githubusercontent.com/…` → `Recv failure: Connection reset by peer`（HTTP 000，0.18 秒）；`curl https://raw.githubusercontent.com/…` → HTTP 200，1.66 秒。即 **https 通、http 不通**——网络策略不对称，而包硬编码了 http。
5. **矫正（已执行）：** 用 https 把两个标志文件下到 `ref/`，显式传入：

```
curl -sS https://raw.githubusercontent.com/ebecht/MCPcounter/master/Signatures/genes.txt -o ref/
MCPcounter_genes.txt
curl -sS https://raw.githubusercontent.com/ebecht/MCPcounter/master/Signatures/probesets.txt -o
ref/MCPcounter_probesets.txt
```

```
genes <- read.table("ref/MCPcounter_genes.txt", sep="\t", stringsAsFactors=FALSE, header=TRUE,
                    colClasses="character", check.names=FALSE)   # 与 MCPcounter.estimate 默认一致
probesets <- read.table("ref/MCPcounter_probesets.txt", sep="\t", stringsAsFactors=FALSE,
                        colClasses="character")                              # 与
MCPcounter.estimate 默认一致
res_mcp <- deconvolute(exprsn, "mcp_counter", genes=genes, probesets=probesets)
```

结果：11 细胞型 × 42 样本，存入 `deconv_methods/res_mcp_counter.tsv`。脚本：`scripts/part3_mcp_counter_fix.R`。即用 https 拉的本地副本替换 http 默认——同一份数据，不改包。

6. **记录的发现：** 11 行输出里 `Monocyte` 与 `Macrophage/Monocyte` 数值**在所有 42 样本上完全相同**。根因是 `immunedeconv::cell_type_map`：MCPcounter 的单一源群 `Monocytic lineage` 被映射成**两个**输出名。故 11 行只有 **10 个独立群**——映射表重复，非计算伪象。

第三节课原样跑时阻塞记入 `mcp_counter_blocked.txt`；该文件现在还记录了随后的矫正执行与重复映射发现。

APP-02 / APP-03 —— EPIC 收敛警告与 xCell/GSVA 弃用警告：均良性，结果照常产出。

CIBERSORT 向 cwd 写文件 —— `deconvolute(exprsn, "cibersort")` 调 CIBERSORT.R 源码，它向当前工作目录写 `CIBERSORT-Results.txt`。因脚本 `setwd()` 到 `04_immune_infiltration/`，文件落在此处（42 样本，已验证）。早期 3 样本探测跑在仓库根留下的散落副本已删除。

结果解读

七种方法在细胞型粒度（TIMER 6 型 → xCell 39 型）与输出尺度上都不同。教材的核心论点——**只有绝对分数法能画堆叠柱**——由两张 quanTIseq 图示范：堆叠柱（`qts_barplot.png`，分数和为 1）对 quanTIseq（绝对法）正确，而点/分面图（`qts_scatterplot.png`，每细胞型自由 y 轴）是对任何方法都安全的选择。`scale='row'` 的热图（`qts_heatmap_row_scaled.png`）是公平的跨样本比较：每个细胞型被 z-score，所以巨噬细胞高的样本读红，与方法单位是分数还是任意值无关。

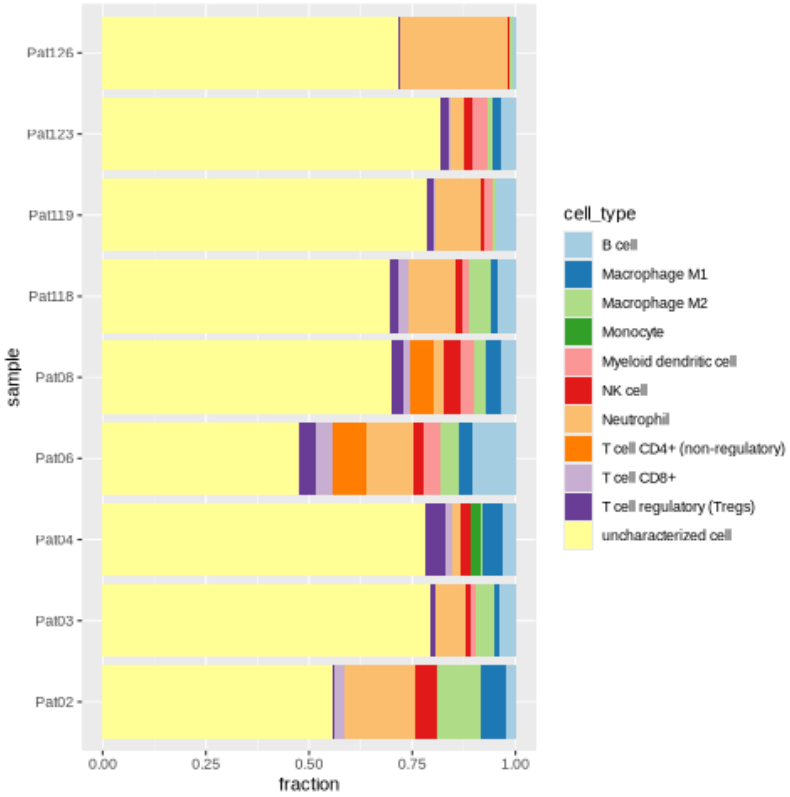


图 5 — `qts_barplot.png`：quanTIseq 的细胞比例堆叠柱，每个样本一根、各段和为 1。

堆叠柱只对”绝对比例”方法合法，而 quanTIseq 正是其中之一——所以这里的几何是诚实的。但图里实际显示的是：每根柱子大部分都是浅色的”未分类细胞”（这些肿瘤里占 60-90%），免疫细胞只是右端那些细条。这是方法的正确输出（肿瘤样本里确实有大量细胞任何 signature 都解释不了），但它让免疫信号在视觉上读不出来——这正是同样的数字要在下面再画两遍的原因。

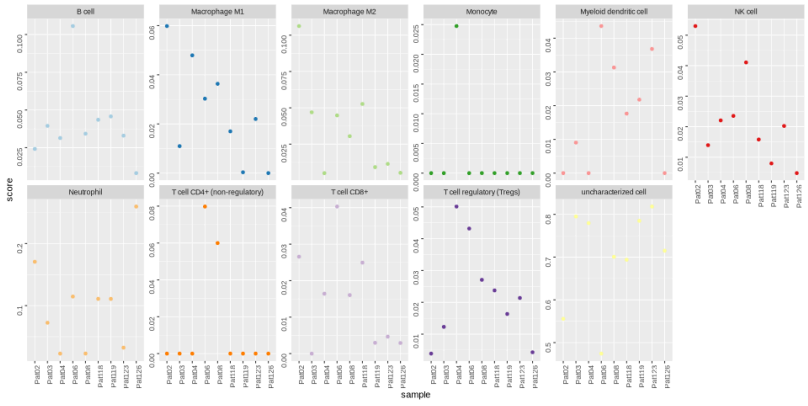


图 6 — `qts_scatterplot.png`：同样的比例画成点/分面图，每个细胞类型独立 y 轴。

任何方法都安全的选择：每个细胞类型一格、各有自己的轴，于是哪怕从不超过 5% 的群体也能看出差别。要比较”同一细胞类型里的不同样本”，读这张；要看”单个样本的组成”，读堆叠柱。

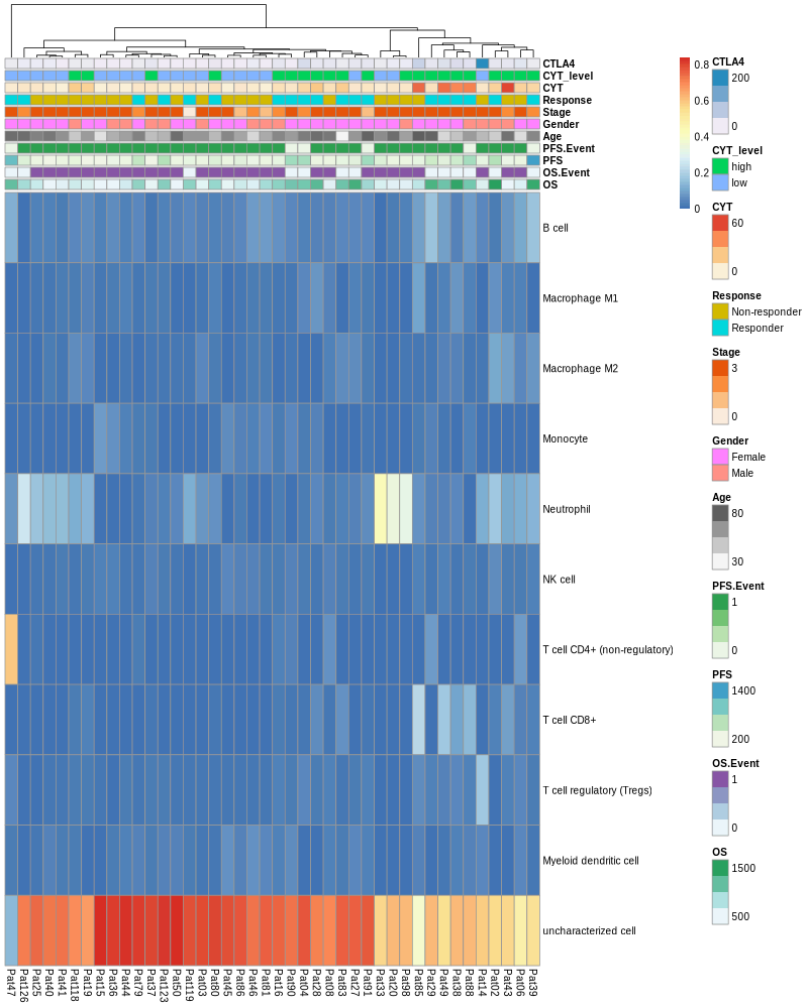


图 7 — `qts_heatmap.png`：未缩放的细胞类型 × 样本矩阵。

故意不缩放。最占色的仍然是未分类那一行（免疫类型里则是巨噬细胞），这是原始结构——但注意低丰度群体（B 细胞、DC、NK）被压成同一种颜色。只看到这张热图的读者会得出”这个队列只有巨噬细胞含量在变，别的没差别”，那是共用色标造成的假象。



图 8 — `qts_heatmap_row_scaled.png`：同一个矩阵按**每个细胞类型**做 z 分数。

公平的跨样本比较，也是 Part 3 里唯一能回答”哪个样本哪种细胞多”的图。按行缩放后每种细胞类型各有自己的轴，于是 B 细胞明明数值很小、该高的样本照样是红的。读它有两个要点：z 分数是**相对**的——红表示”相对本队列偏高”，不是绝对偏高；在 42 个样本上做行缩放会让单点离群看起来很夸张，所以任何惊人的行都要回到图 6 交叉核对。

参数调优

- 本地标志集修复已应用；`ref/MCPcounter_genes.txt` 与 `ref/MCPcounter_probesets.txt` 已入库，重跑完全离线。
- `config_env$xcell_cores` 控制 xCell 并行；大队列可调高。
- `timer` 的癌种务必对照 `timer_available_cancers` 核对——错癌种会静默加载错参考。

扩展

- 七法全跑后，对各方法的 T 细胞估计做相关热图比较——去卷积论文常见的合理性检查。
- `immunedeconv` 还支持 `consensus_tme`、`abis`、`estimate`（见 `deconvolution_methods`）——教材未用但可用。

6. Part 4 —— TIDE（代码块 27–33）

原文

```
tidepy .../VanAllen2015_CTLA4_Melanoma_RNASeq -o /path_to_your_folder/res_tide.txt -c Melanoma --
force_normalize
```

```
res_tide <- read.table('.../res_tide.txt', sep='\t', header=TRUE, row.names=1)
colnames(res_tide)[2] <- 'Predicted Responder'
res_tide <- res_tide[rownames(metadata),]      # 按 metadata 重排
# factor 重标签: Predicted Responder False/True -> Non-responder/Responder
#                CTL.flag False/True -> CTL Low / CTL high
#                cbind metadata$Response 作第 15 列 'Responder'
# heatmap(t(res_tide_heatmap), scale='row', annotation_col=annotation)
# 柱图: 每患者 TIDE 分，按 CTL.flag 分面，按预测/真实应答着色
library(pROC)
TIDE_roc <- roc(Responder ~ TIDE, data=res_tide, aur=TRUE)
ggroc(TIDE_roc, legacy.axes=TRUE) + geom_segment(...)   # AUC 标注
TIDE_roc_smooth <- roc(Responder ~ TIDE, data=res_tide, aur=TRUE, smooth=TRUE)
# … 平滑 ROC …
```

原理

- **`tidepy`**（shell）跑 TIDE 模型（Jiang 等 2018 Nature Medicine），从治疗前表达预测免疫治疗应答：**TIDE 分 < 0 预测应答，> 0 预测无应答**。`-c Melanoma` 选黑色素瘤癌种模型；`--force_normalize` 强制 log2(x+1) + 中心化（APP-04 的三行 `[WARN]` 记录的正是此操作）。输出列：`No benefits`、`Responder`、`TIDE`、`IFNG`、`MSI Score`、`CD274`、`CD8`、`CTL.flag`、`Dysfunction`、`Exclusion`、`MDSC`、`CAF`、`TAM M2`、`CTL`。
- **`res_tide[rownames(metadata),]`** 重排到 metadata，使热图注释对齐。
- **`roc(Responder ~ TIDE, data=res_tide)`**：pROC 的公式接口。`Responder`（第 15 列，真实临床应答）是 factor，levels `Non-responder` < `Responder`；pROC 设 `controls = Non-responder, cases = Responder`，方向 `controls > cases`（即 **TIDE 越低越应答**）—— 与 TIDE < 0 规则一致。
- **`ggroc(..., legacy.axes=TRUE)`** x 轴画 1−specificity（曲线凸向左上）；`geom_segment(0,0 → 1,1)` 是机会对角线。
- **`smooth=TRUE`** 对 ROC 曲线做双正态平滑。

我们跑了什么

- **tidepy：** 墙钟 11 秒；输出 `res_tide.txt` 10,259 B（与教师预跑副本 `/lustre1/share/rnaimmune/res_tide.txt` 字节同尺寸），42 样本 × 14 列。三行预期 `[WARN]`（APP-04）。
- **ROC：** AUC = 0.7872（原始）；AUC = 0.7718（平滑）。

产出：`res_tide.txt`、`tide_heatmap.png`、`tide_predicted_responder.png`、`tide_responder.png`、`roc.png`、`roc_smooth.png`。

坑

- **APP-04** —— 三行 `[WARN]` 在 `--force_normalize` 加 CPM 尺输入下**预期**，非报错。5.73 % 基因缺失是 Entrez ID 转换损失（可接受，<10 %）；>80 % 阳性表达警告正是触发归一化的原因。
- 两张柱图仅在着色美学上不同：`tide_predicted_responder.png` 按 **TIDE 预测**着色，`tide_responder.png` 按**真实临床应答**着色——对比二者即可目视读预测 vs 真实。
- `roc(..., aur=TRUE)` —— 注意教程代码参数是 `aur`（非 `auc`）；pROC 接受此别名，AUC 存于 `$auc`。一个能跑通的笔误。

结果解读

AUC ≈ 0.79（原始）/ 0.77（平滑）意味着 TIDE 在本队列能较好区分应答与无应答——与 TIDE 原文黑色素瘤验证一致。TIDE < 0 规则与 ROC 方向吻合。柱图显示多数预测应答者落在 CTL-high 分面（T 细胞在场），多数无应答者在 CTL-low，体现 TIDE 的双轴逻辑（CTL-high 走功能失调轴，CTL-low 走排除轴）。

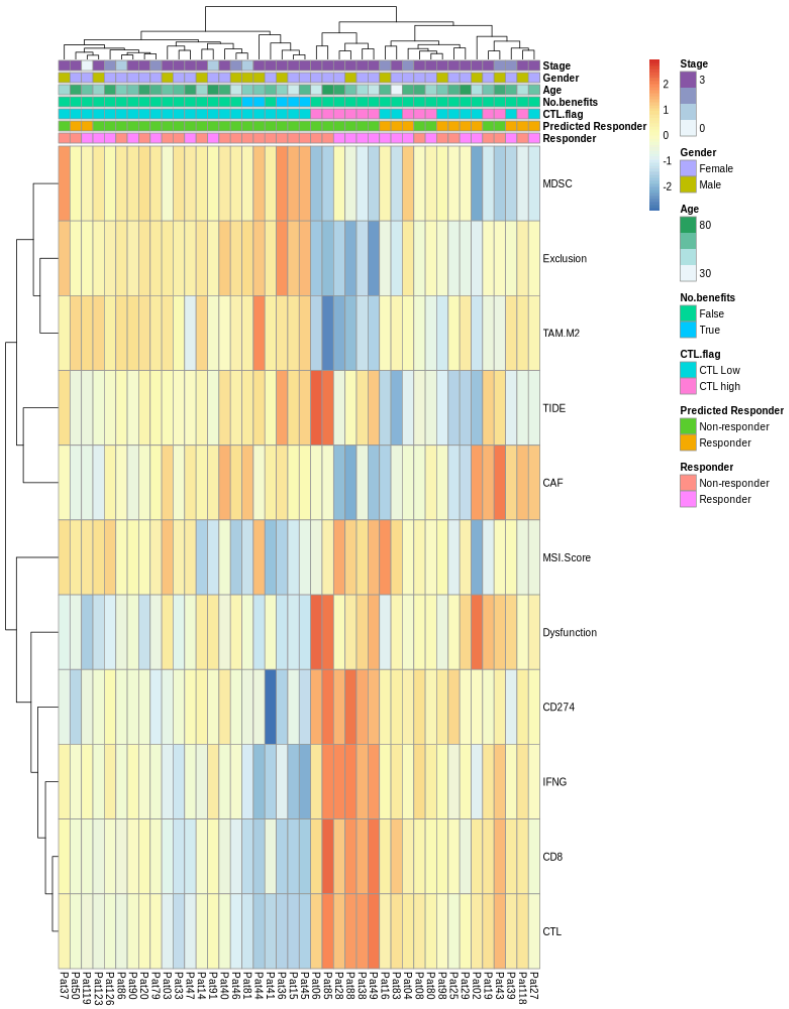


图 9 — `tide_heatmap.png`：TIDE 输出矩阵（TIDE、IFNG、MSI、CD274、CD8、Dysfunction、Exclusion、MDSC、CAF、TAM M2、CTL…）按行缩放的热图，列注释是临床元数据。

11 个 TIDE 衍生特征 × 42 个病人，按行 z 分数，让颜色比较的是病人而不是特征（TIDE 与 MSI 不在同一个量纲上）。对着注释条竖着读：CTL/Dysfunction/Exclusion 这一块就是两轴故事——CTL 高且 Dysfunction 高的病人，是 TIDE 认为对检查点阻断有反应的那批；CTL 低且 Exclusion 高的，是它认为无论如何都会失败的那批。注意 `Responder`（真实临床标签）也画在同一条注释带上，这正是下面两张图可读的前提。

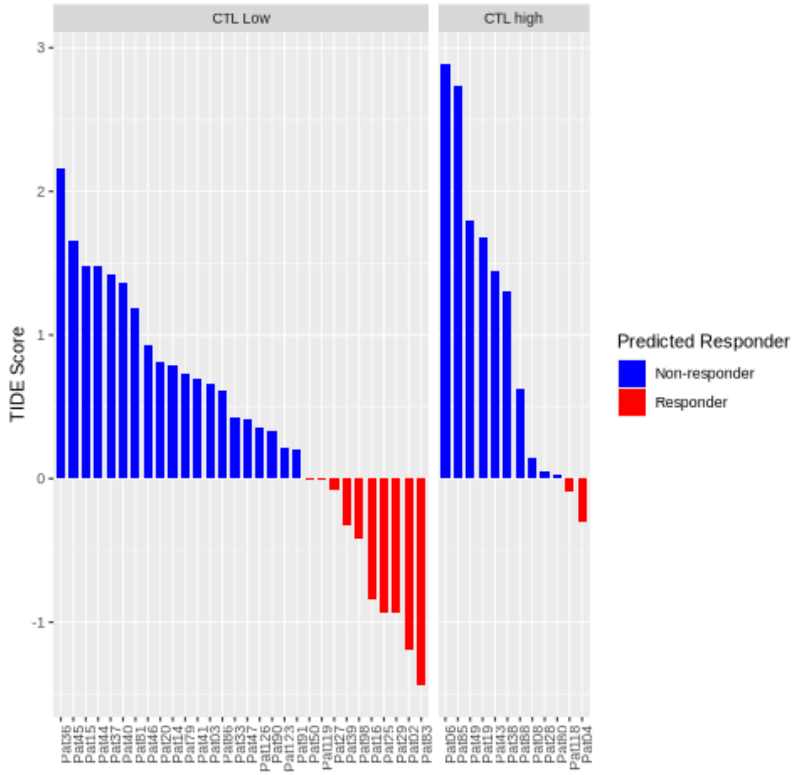


图 10 — `tide_predicted_responder.png`：每个病人的 TIDE 分数，按 CTL 状态分面，按 TIDE 的**预测**着色。

规则是 `TIDE < 0 → 预测有反应`，而分面让这条规则有意义：CTL 高的分面里多数柱子为负（预测响应），CTL 低的分面里多数为正。蓝/红填充是符号的机械结果，所以这张图其实是”阈值长什么样”的照片——用来挑出那些贴着 0 的病人，他们的判定会随 `--vthres` 的一点改动而翻转。

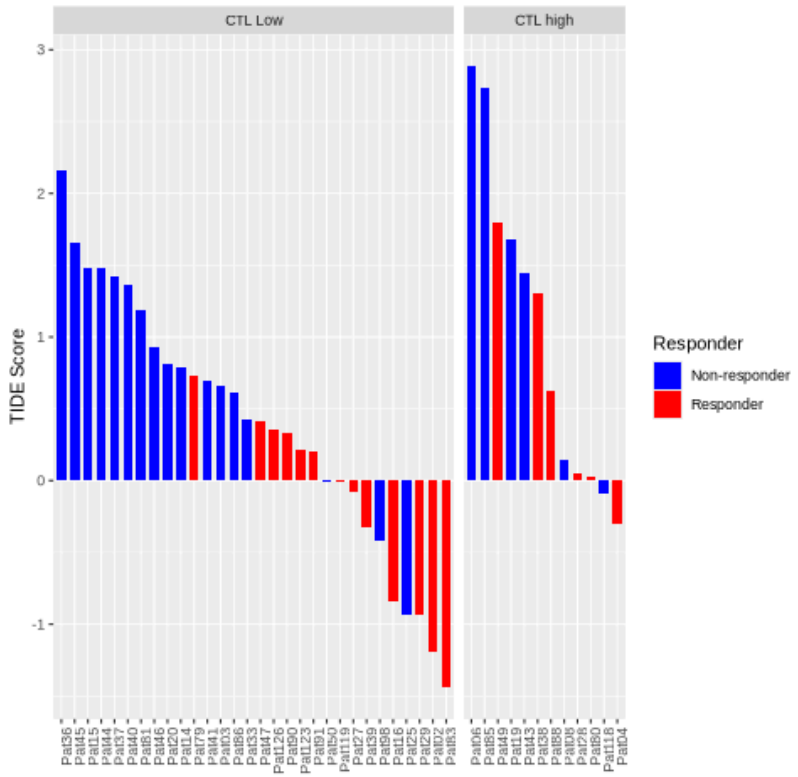


图 11 — `tide_responder.png`：同样的柱子，改按**真实**临床反应着色。

把它和图 10 并排放就是画两张的全部意义：填充在少数病人上不一致，而这些不一致正是下面 ROC 概括掉的错误。并排还能最快看出失败模式——几个真实响应者落在高（正 TIDE）的位置，即 TIDE 漏掉了他们，而不是反过来。

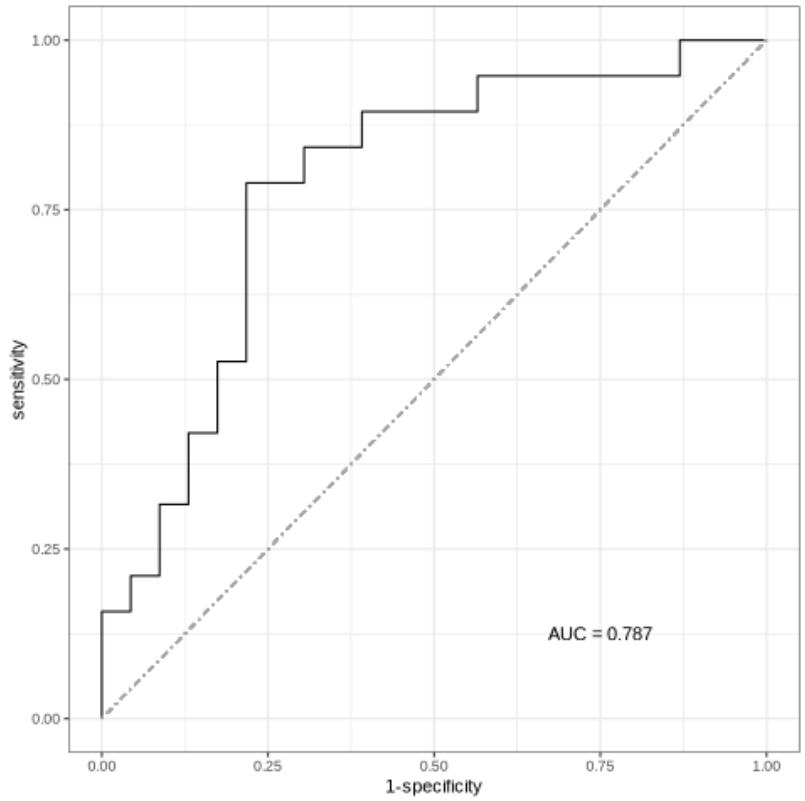


图 12 — `roc.png`：TIDE 预测临床反应的 ROC（原始曲线，AUC = 0.787）。

曲线呈阶梯状，因为 42 个病人只有 42 个阈值——每一级台阶就是一个病人在越过切点。阶梯是样本量造成的，不是模型性质；能读的只有它相对虚线的位置：AUC 0.787 意味着随机取一个响应者和一个非响应者，TIDE 有 79% 的概率把两者排序排对。注意坐标约定：x 是 1 − 特异度（向右升高），所以**好**模型向左上凸，而”TIDE 越低越可能响应”这个方向假设已经被 pROC 预先定好了。

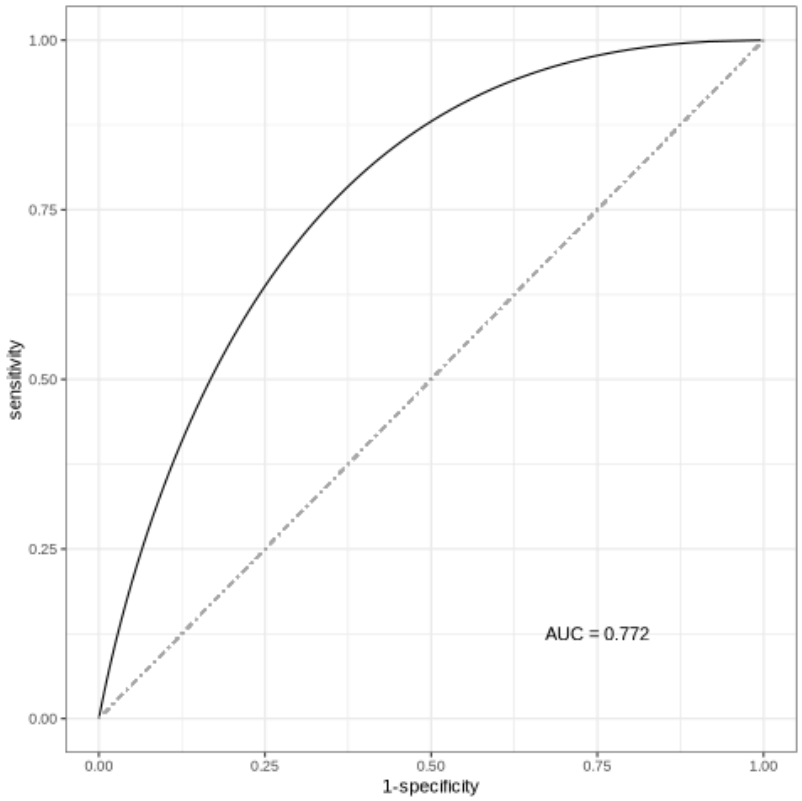


图 13 — `roc_smooth.png`：同一张 ROC 做双正态平滑（AUC = 0.772）。

平滑用诚实的阶梯换来一条可跨模型比较的曲线——代价是 0.015 的 AUC，而这恰恰是最有用的一句话：42 个病人的原始 AUC 偏乐观，正因为它是在同 42 个点上拟合出来的阶梯。报这个数时，用平滑值作为保守估计、用 `ci.auc()` 给出区间；只写一句”AUC 0.79”在这个样本量下是一个邻域很宽的点估计。

参数调优

- `--vthres` 可设自定义 TIDE 阈值（默认 0.0）；调高以特异性换敏感性。
- `--pretreat` 改变既往免疫治疗后进展患者的应答规则——此处不适用（ipilimumab 为一线）。
- 要 AUC 置信区间：`ci.auc(TIDE_roc)`。

扩展

- 把 TIDE 的 `Dysfunction` 与 `Exclusion` 分数与 Part 1 的 CYT、CTLA4 相关对照——连贯叙事：高 CTL → 功能失调轴，低 CTL → 排除轴。
- 把 TIDE 预测与 Part 3 的 `immunedeconv` CD8/T 细胞估计交叉对照。

7. 检查清单、产物清单与总结

检查清单

编号	检查项	结果
ck1	表达矩阵加载，CPM 列和 = 1e6	✓ 42 × 1e6
ck2	metadata 按 exprsn 重排	✓ `all(rownames(metadata)==colnames(exprsn))` TRUE
ck3	CYT 中位数分组	✓ 21 high / 21 low，无并列
ck4	KM 图 + 风险表渲染	✓

编号	检查项	结果
ck5	Cox 模型收敛（6 协变量，29 事件）	✓ Concordance 0.679
ck6	森林图渲染	✓
ck7	七种去卷积方法完成	✓（MCPcounter 已矫正——APP-01 矫正已应用）
ck8	quanTIseq rmgenes 变体覆盖首次调用	✓ 11 × 42
ck9	堆叠柱 / 点图 / 热图 / 按行缩放热图均渲染	✓
ck10	tidepy 完成，输出字节尺寸与教师一致	✓ 10,259 B
ck11	res_tide 按 metadata 重排	✓
ck12	ROC + 平滑 ROC 渲染带 AUC 标注	✓ 0.787 / 0.772
ck13	全部产出在 `results/` 按步骤，仓库无原始数据	✓

产物清单（31 文件，无原始数据）

```
02_gene_correlation/   GZMA_cor_PRF1.png, CTLA4_cor_CYT.png, scripts/part1_gene_correlation.R
03_survival_analysis/  survival_CYT.png, cox.png, scripts/part2_survival_analysis.R
04_immune_infiltration/ qts_barplot.png, qts_scatterplot.png, qts_heatmap.png,
                        qts_heatmap_row_scaled.png, CIBERSORT-Results.txt,
                        mcp_counter_blocked.txt, log_part3.txt,
                        deconv_methods/{res_mcp_counter,res_epic,res_quantiseq,
                        res_quantiseq_rmgenes,res_xcell,res_cibersort,
                        res_cibersort_abs,res_timer}.tsv,
                        scripts/part3_immune_infiltration.R,
                        scripts/part3_mcp_counter_fix.R
05_tide/               res_tide.txt, tide_heatmap.png, tide_predicted_responder.png,
                        tide_responder.png, roc.png, roc_smooth.png,
                        log_tidepy.txt, scripts/run_tidepy.sh, scripts/part4_tide_analysis.R
```

总结

第三节课是纯下游分析——无 FASTQ、无比对、无 SLURM。四个 Part 全在登录节点用 R 4.3.3（加一行 `tidepy` shell）跑完，约 6 分钟。七条已验证坑入档；其中一条（APP-01，MCPcounter）在原样跑时是硬阻塞（conda MCPcounter 1.1.0 把标志基因集硬编码成 **http** GitHub 拉取，登录节点到 GitHub 的 http 被重置）——矫正随后执行并验证（本地 https 标志集作为显式参数传入），故七种去卷积方法现在都有结果。记录了一个重复映射发现：MCPcounter 的 11 行输出里 `Monocyte` 与 `Macrophage/Monocyte` 数值完全相同，因为 immunedeconv 把单一源群 `Monocytic lineage` 映射成两个名（10 个独立群，非 11）。TIDE 输出与教师预跑副本字节同尺寸，ROC AUC（0.79）复现了原文黑色素瘤区分度。

附录 A. 偏差日志（本次，精简但忠实原文）

- **交互式 → Rscript：** 每个 Part 写成 `partN_*.R`，在 `LC_ALL=C`/`LANG=C` 下用 `Rscript` 跑（RNA-seq R 段 segfault 规避，项目记忆）。代码内容逐字取自教材；包装仅 `setwd()` + 诊断 `cat()`/`print()`。
- **跨 Part 对象共享：** 教材在一个交互式 R 会话里让 `exprsn`、`sig`、`metadata`、`res_qts` 跨代码块流动。脚本化为四个进程后，每个脚本重新派生所需对象（重读表达矩阵、重算 CYT）。代码内容不变，仅加载重复。
- **Part 4 输入路径：** 教材读教师预跑的 `/lustre1/share/rnaimmune/res_tide.txt`。本次改读自己 `tidepy` 跑出的 `results/05_tide/res_tide.txt`（字节同尺寸）。教材代码块 28 的注释明确把此作为”自己跑 tidepy 时的预期用法”。
- **MCPcounter（APP-01）：** 第三节课原样跑时包进 `tryCatch` 并记录阻塞；随后的矫正（`scripts/part3_mcp_counter_fix.R`）用本地 https 标志集跑出，写入 `deconv_methods/res_mcp_counter.tsv`。`mcp_counter_blocked.txt` 同时记录原始阻塞与已应用的矫正。
- **去卷积结果落盘：** 教材只 `print()` `res_qts`；本次额外把每方法矩阵写 `deconv_methods/*.tsv`（附加、非破坏）。

- **CIBERSORT-Results.txt**：早期 3 样本探测跑在仓库根留下的散落副本已删；`04_immune_infiltration/` 里的 42 样本正式副本保留。

附录 B. 复现

```
source /lustre1/share/miniconda3/etc/profile.d/conda.sh && conda activate rnaimmune
export LC_ALL=C; export LANG=C
cd /gpfs1/home/bjx131_pkuhpc/class/RNA-seq-application/results
Rscript 02_gene_correlation/scripts/part1_gene_correlation.R
Rscript 03_survival_analysis/scripts/part2_survival_analysis.R
Rscript 04_immune_infiltration/scripts/part3_immune_infiltration.R   # ~2 分钟，MCPcounter 阻塞
(APP-01)
bash    05_tide/scripts/run_tidepy.sh                                # ~11 秒
Rscript 05_tide/scripts/part4_tide_analysis.R
# APP-01 矫正（离线，需 ref/MCPcounter_*.txt）：
Rscript 04_immune_infiltration/scripts/part3_mcp_counter_fix.R        # ~10 秒
```

所有产出落到对应步骤目录（脚本 `setwd()` 进去）。原样跑时 MCPcounter 预期以 `Connection reset by peer` 失败（记入 `mcp_counter_blocked.txt`）；矫正脚本随后用入库的 `ref/MCPcounter_*.txt` 标志集离线跑出。